

JAVA Programming (CST015)

Lecture 4: Methods in JAVA



Department of Computer Science and Engineering
National Institute of Technology, Srinagar, Jammu and Kashmir
March 09, 2026

Methods in JAVA

- A **method** is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a method.
- Methods are used to perform certain actions, and they are also known as **functions**.
- **Why to use methods?** To reuse code: define the code once, and use it many times.

Creating a Method

- A method must be declared within a class.
- It is defined with the name of the method, followed by parentheses ().
- Java provides some pre-defined methods, such as `System.out.println()`,
- but you can also create your own methods to perform certain actions.

Type of Methods

1. Predefined Method:

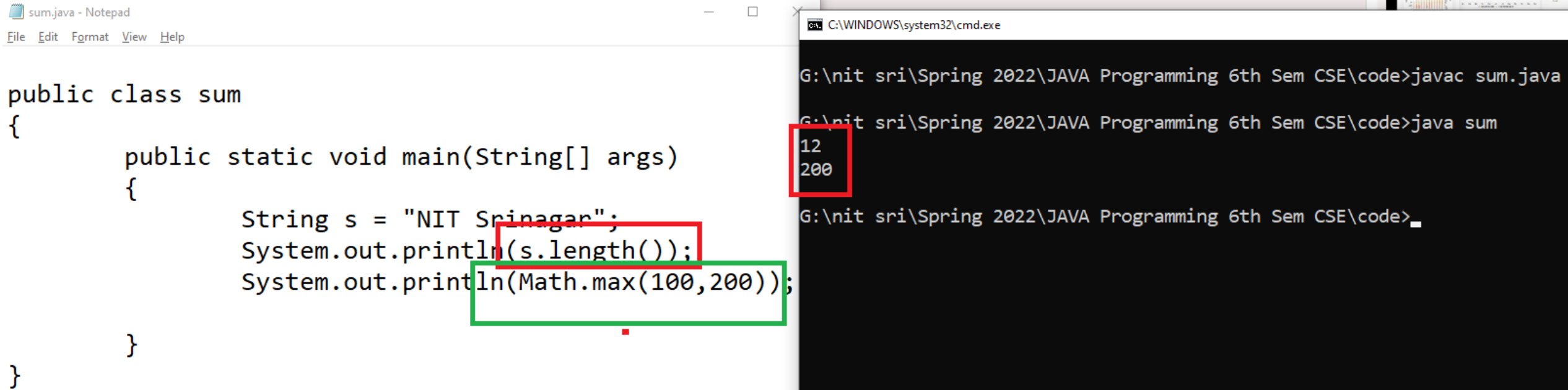
- In Java, predefined methods are the method that is already defined in the Java class libraries.
- Predefined Methods are also called as **built-in methods**.
- these methods can be used just by calling them in the program at any point.
- Some pre-defined methods are **length()**, **equals()**, **compareTo()**, **sqrt()**, **max()**, etc.
- Each and every predefined method is defined inside a class. Such as **print()** method is defined in the **java.io.PrintStream** class. It prints the statement that we write inside the method.

2. User-defined Method:

- The method written by the user or programmer is known as a **user-defined** method.
- These methods are created according to the requirement.

Built-in methods

- main(), length(), max() are the built-in methods.
- For example, max() method is already defined in Math class.
- length() class is defined in String Class.



```
sum.java - Notepad
File Edit Format View Help

public class sum
{
    public static void main(String[] args)
    {
        String s = "NIT Spinagar";
        System.out.println(s.length());
        System.out.println(Math.max(100,200));
    }
}
```

```
C:\WINDOWS\system32\cmd.exe

G:\nit sri\Spring 2022\JAVA Programming 6th Sem CSE\code>javac sum.java
G:\nit sri\Spring 2022\JAVA Programming 6th Sem CSE\code>java sum
12
200
G:\nit sri\Spring 2022\JAVA Programming 6th Sem CSE\code>
```

Creating a User-defined Method

1. Method Declaration
2. Method Definition
3. Method Calling

Creating a Method:

Method Declaration and Definition

```
public class Main
{
    static void myMethod()                // Declaration and Definition
    {
        // code to be executed
    }
}
```

- **myMethod()** is the name of the method
- **static** means that the method belongs to the Main class and not an object of the Main class.
- **void** means that this method does not have a return value.

Call a Method

- To call a method in Java, write the method's name followed by two parentheses () and a semicolon;

```
myMethod();
```

A method can also be called multiple times

Method Example

```
public class Main {  
  
    static void myMethod()  
    {  
        System.out.println("i am inside method");  
    }  
  
    public static void main(String[] args) {  
        myMethod();  
    }  
}
```

Output



PracticeQuestions



File

Edit

View

```
class Main {
    public static void main(String[] args) {
        int x = 10;
        System.out.println("Before: x = " + x);
        increment(x);
        System.out.println("After: x = " + x);
    }

    public static void increment(int x) {
        x++; // Incrementing the parameter
        System.out.println("Inside increment method: num = " + x);
    }
}
```

Java passes arguments by value, where a copy of the value of the variable is passed to the method.

In Java, when you pass a primitive type or an object reference as an argument to a method, **a copy of the value is passed, not the original variable itself.**

This means that **any modifications made to the parameter inside the method do not affect the original variable outside the method (for primitives).**

In Previous Slide Code:

Initially, x is 10.

When **increment(x)** is called, a copy of the value of **x** (which is 10) is passed to the **x** parameter inside the increment method.

Inside the increment method, **x** is incremented to 11, but this change does not affect the original variable **x** outside the method.

Overloading the main method

Overloading of main() method can be done in java.

We can define any number of main() method in the class, but the method signature must be different

Overloading of main() method

```
class OverloadMain
{
    public static void main(int a) //overloaded main method
    {
        System.out.println(a);
    }
    public static void main(String args[])
    {
        System.out.println("main method incoked");
        main(6);
    }
}
```

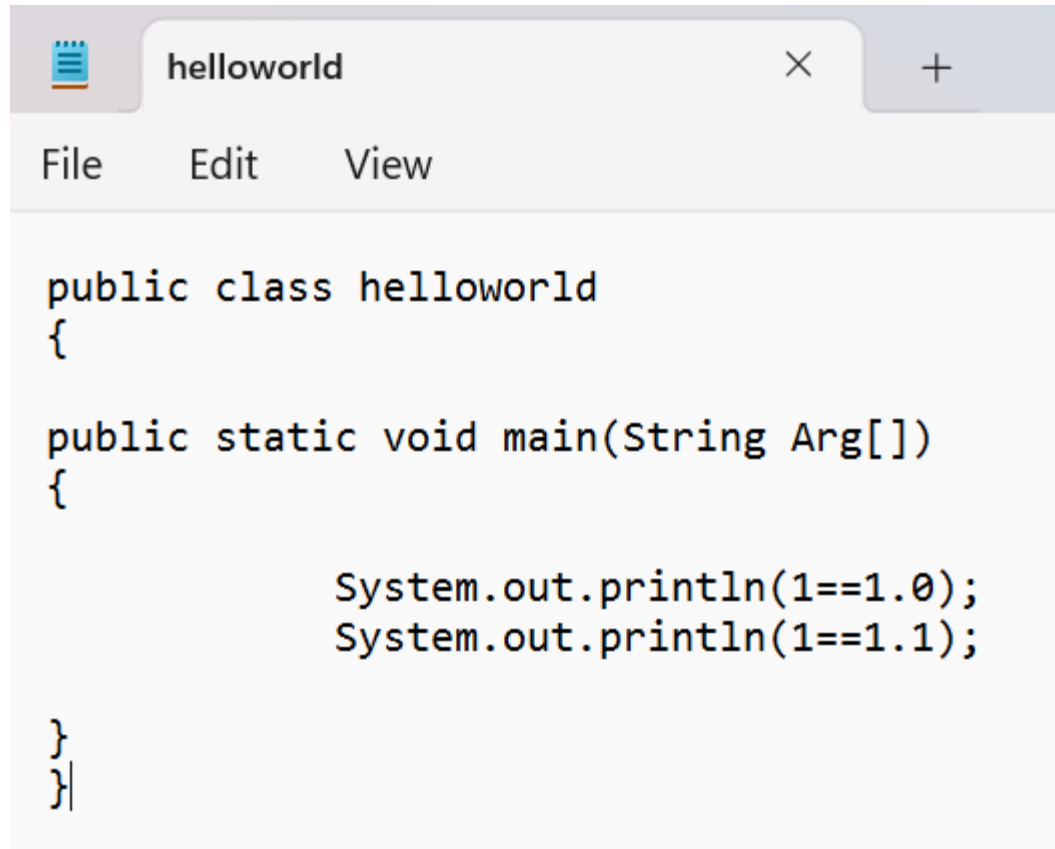
Output?

```
class OverloadMain_return
{
    public static int main(String args[])
    {
        return 10;
    }
    public static void main(String num[])
    {
        System.out.println("main method invoked");
    }
}
```

Exercise:

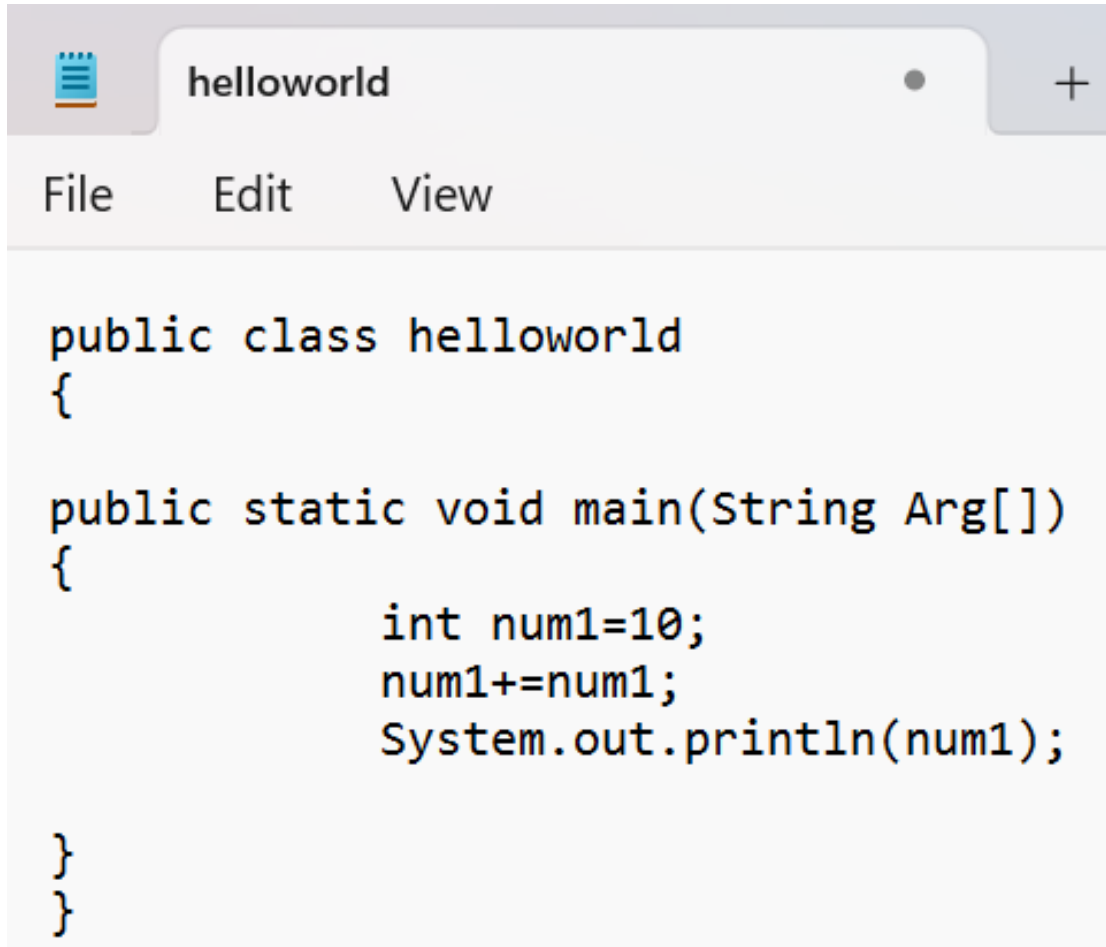
- WAP using functions (methods) to find the sum of two numbers.

Output?



```
public class helloworld
{
    public static void main(String Arg[])
    {
        System.out.println(1==1.0);
        System.out.println(1==1.1);
    }
}
```


Output?

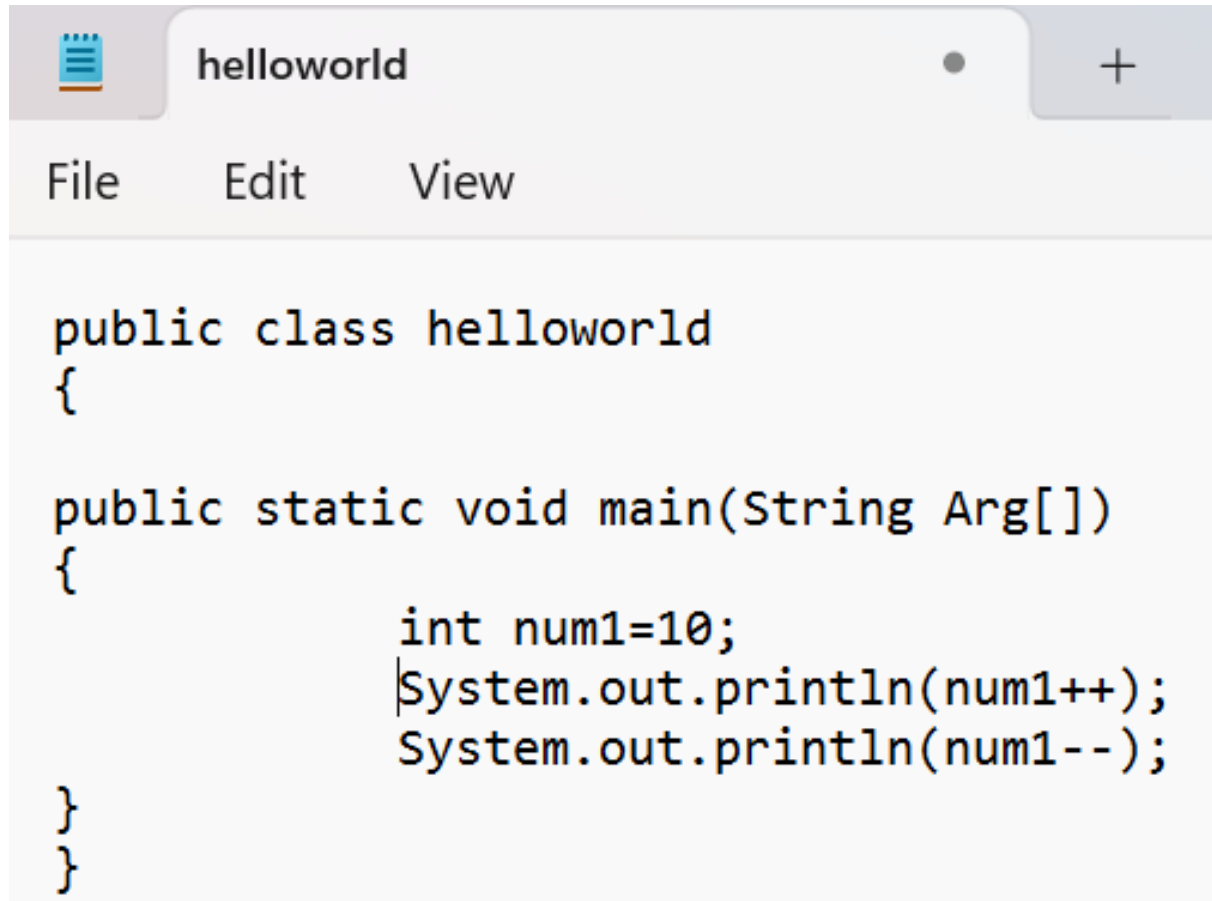


```
public class helloworld
{

public static void main(String Arg[])
{
    int num1=10;
    num1+=num1;
    System.out.println(num1);

}
}
```

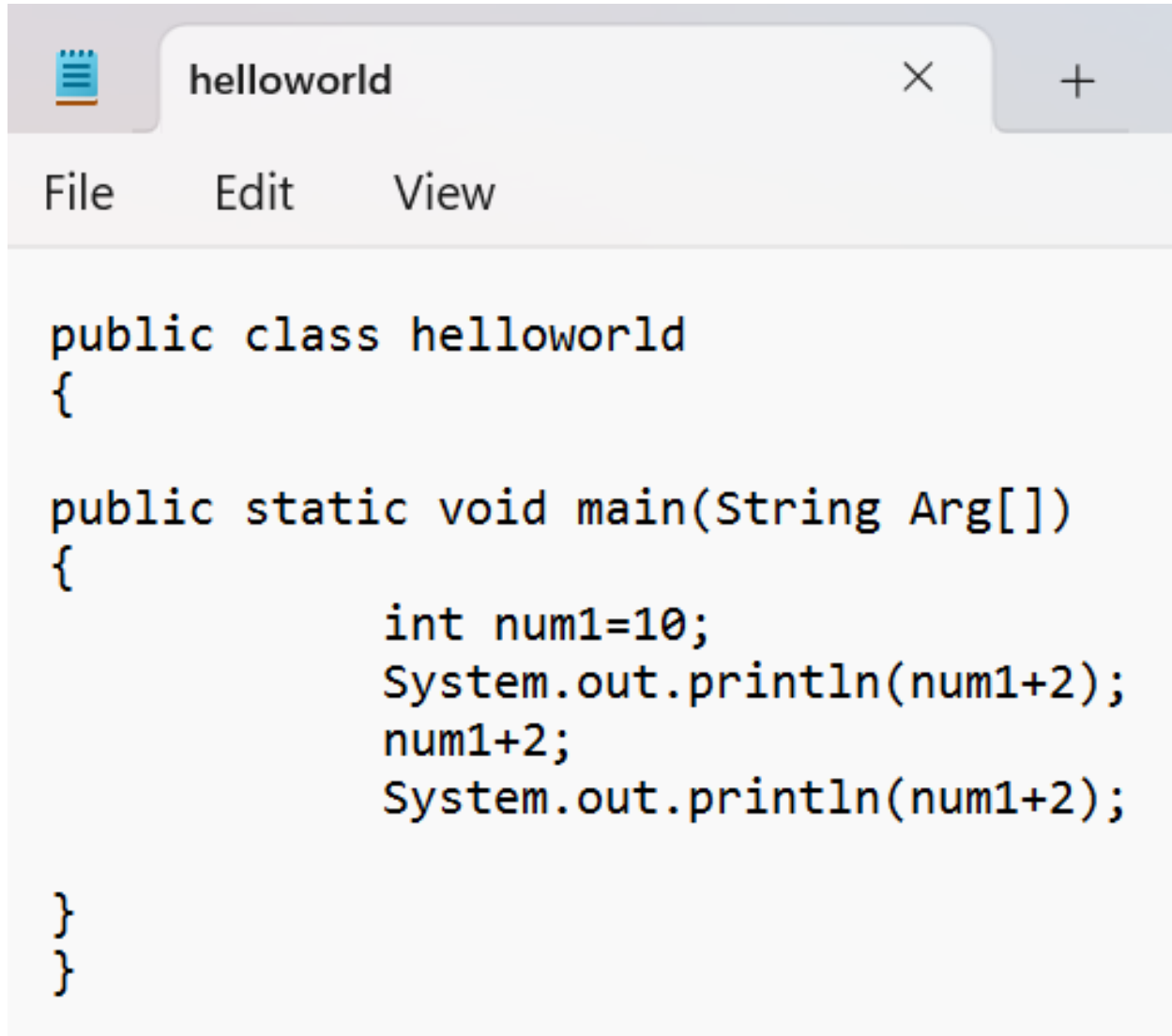
Output?



```
public class helloworld
{

public static void main(String Arg[])
{
    int num1=10;
    System.out.println(num1++);
    System.out.println(num1--);
}
}
```

Output?



The image shows a screenshot of a code editor window. The window has a title bar with a tab labeled 'helloworld' and standard window controls (close, maximize, and a plus sign for new tabs). Below the title bar is a menu bar with 'File', 'Edit', and 'View' options. The main area of the window contains the following Java code:

```
public class helloworld
{

public static void main(String Arg[])
{
    int num1=10;
    System.out.println(num1+2);
    num1+2;
    System.out.println(num1+2);

}
}
```